Let’s assume we only have **3 FAQs** to keep it simple.

Index	Question
0	What is Frontend?
1	Languages?
2	Frameworks?

Initially,

```
1. showIndex = null
```

Phase 1 : First Render

React enters `Faq` .

Step	Code	Value
1	`useState(null)`	`showIndex = null`
2	Creates `handleQnA(index)`	Function created
3	Starts `map()`	Loop starts

First iteration

```
1. index = 0
```

React reaches

```
1. showAns={index === showIndex}
```

Calculate

```
1. 0 === null
```

Result

```
false
```

Now this line

```
handleQnA={() => {
  handleQnA(index);
}}
```

React creates a function.

Think of it as

```
function child0Function() {
  handleQnA(0);
}
```

React passes

Prop	Value
qnaData	Question 0
showAns	false
handleQnA	child0Function

to first child.

Second iteration

```
index = 1
```

Calculate

```
1 === null
```

↓

```
false
```

Create another function

```
function child1Function() {
```

```
    handleQnA(1);
}
```

Pass to second child.

Third iteration

```
index = 2
```

Calculate

```
2 === null
```

↓

```
false
```

Create

```
function child2Function() {
    handleQnA(2);
}
```

Pass to third child.

Parent has now created

Child	showAns	handleQnA received
0	false	`() => handleQnA(0)`
1	false	`() => handleQnA(1)`
2	false	`() => handleQnA(2)`

Notice something…

Every child gets **its own function**.

Phase 2 : Child Render

First child receives

```
const Qna = ({
    qnaData,
    showAns,
    handleQnA
})
```

So for first child

```
qnaData = Question0

showAns = false

handleQnA = child0Function
```

React renders

```
<span onClick={handleQnA}>
```

means

```
<span onClick={child0Function}>
```

Second child

```
<span onClick={child1Function}>
```

Third child

```
<span onClick={child2Function}>
```

Browser looks like

```
+ What is Frontend?

+ Languages?

+ Frameworks?
```

No answers.

Phase 3 : User clicks Second Question

User clicks

```
1. Languages?
```

Browser sees

```
1. onClick={child1Function}
```

So browser executes

```
1. child1Function();
```

Now run it.

```
1. function child1Function(){
2.    handleQnA(1);
3. }
```

Now run

```
1. handleQnA(1);
```

Parent function executes

```
1. const handleQnA=(index)=>{
2.    setShowIndex(index);
3. }
```

Here

```
1. index = 1
```

So

```
1. setShowIndex(1);
```

Now React changes

```
1. Before
```

```
2.
3. showIndex = null
```

↓

```
1. After
2.
3. showIndex = 1
```

Phase 4 : Re-render

Whenever state changes,

React starts again from top.

Again

```
1. Faq()
```

runs.

Now

```
1. showIndex = 1
```

instead of

```
1. null
```

First iteration

```
1. index = 0
```

Calculate

```
1. 0===1
```

↓

```
1. false
```

Second iteration

```
1. index=1
```

Calculate

```
1. 1===1
```

↓

```
1. true
```

Third iteration

```
1. 2===1
```

↓

```
1. false
```

Now parent sends

Child	showAns
0	false
1	true
2	false

Child receives

Second child

```
1. showAns = true
```

Now React reaches

```
1. {showAns && qnaData.answer}
```

This becomes

```
true && answer
```

↓

```
answer
```

React shows it.

Final Screen

```
+ What is Frontend?

+ Languages?
 HTML CSS JavaScript....

+ Frameworks?
```

Complete Flow in One Table

Step	What happens
1	`showIndex = null`
2	Parent loops through FAQs
3	Each child gets its own `handleQnA` function (`() => handleQnA(0)` , `() => handleQnA(1)` , etc.)
4	User clicks second question
5	Browser executes second child's function
6	Function calls `handleQnA(1)`
7	Parent executes `setShowIndex(1)`
8	React re-renders
9	Only `index === 1` becomes `true`
10	Second answer is displayed

⭐ **tip:** If you can explain this sentence, you've understood the pattern:

> ***Parent owns the state (`showIndex`). Each child receives a callback function. When a child is clicked, it calls that callback, the parent updates its state, React re-renders, and only the matching child gets `showAns = true` .***

That’s exactly how “lifting state up” works in React.